

5. **Fields:** This section, which you can find on the right side of the **Report** view, does a simple but fundamental job. It carries a list of all the columns available in the different tables of the data model so that you can easily drag and drop the columns onto the filters or the properties section. When you click on any column, the **Column tools** ribbon will appear at the top of the window. You can use it to change the data type of a column or set its default aggregation method (like *Sum*, *Count*, or *Average*): this is helpful when you want to fine-tune the behavior of some columns before using them in your visuals.

We made it! We have gone through the four fundamental steps required to build a dashboard in Power BI and used this as an opportunity to have a short tour of the main functionalities of the software.

Let's summarize what we have just seen: everything started with loading some raw data into Power BI, by selecting and configuring the right connectors available in the **Get Data** window. Leveraging the rich set of functionalities of **Power Query Editor**, we took our raw data to the next level and applied an ordered series of transformation steps, which brought us to a number of tables with the right shape and format (which can be previewed at any point using the spreadsheet-like **Data view**). At this point, we used the **Model view** to connect all these tables together, defining the relationships across matching columns and building a single data model. Finally, we opened the **Report view**, where we found all the ingredients (tables and visuals) to build our dashboard. It's time to see this process in action as we build a full Power BI dashboard from scratch in the following tutorial.

Tutorial: Sales Dashboard

This tutorial will take us back to a familiar business, which we have by now grown fond of: the UK-based online retailer selling all-occasions gifts. Automating the weekly Excel reports using KNIME (something we did back in *Chapter 3, Transforming Data*) has granted us quite a reputation that is now pulling us back into the spotlight. As the business keeps growing, the hunger to make descriptive analytics available to a greater number of employees expands. Updating some Excel reports that answer a static list of given questions is not sufficient anymore. We would like to offer the opportunity to deep-dive into quarter-by-quarter and country- and product-level details so that our colleagues can "explore" data by themselves and find interesting insights.

With the help of the financial analyst (who is still full of gratitude for us) and after interviewing a few managers about their most recurrent needs, we are ready to synthesize the requirements expected by each group of users as follows:

- All managers should be able to see at a glance the relative weight of categories and subcategories out of total revenues and see the evolution of sales over time within each product group
- Marketing managers should be able to look at domestic (generated in UK) and international sales separately, having the opportunity to deep-dive by country and category and plan for country-specific digital marketing campaigns accordingly
- Category managers should be given the opportunity to deep-dive by category and spot the articles generating the most sales for each quarter and country

The data follows the same model we encountered in *Figure 3.10* and is made available to us in three separate files: `SalesDashboard-CustomerMD.csv` and `SalesDashboard-ProductMD.csv` include, respectively, the customer and the product master data tables, while `SalesDashboard-Transactions.xlsx` contains the latest transactions (this time, all included in one single file).

It's time to get Power BI Desktop up and running on our computer and start with the first phase of any dashboard creation: loading the required data:

1. As a first step, we load the Customer Master Data stored in a text, comma-separated value file. To do so, click on the **Get Data** icon, available on the **Home** ribbon tab, then select **Text/CSV** as a source type, and click on **Connect**. You will be prompted with a dialog from which you can choose the file to use as a source, in our case, `SalesDashboard-CustomerMD.csv`. At this point, you will see a preview of the table recorded in the file (*Figure 6.13*). Power BI has correctly recognized its format: there are two columns, with the header shown on top in bold. This is looking good, and there is no need to change any setting (you can see, for example, that the comma is rightly used as a column delimiter), so we can click on **Load** to complete the step:

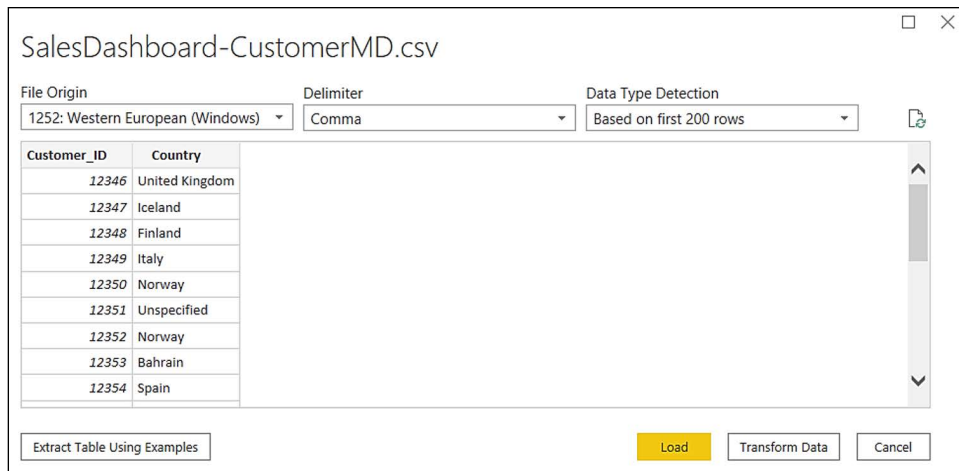


Figure 6.13: Preview of the Customer Master Data source.
You can revise the import settings from the menus in the window

- We can repeat the same procedure (**Get Data** and then **Text/CSV**) to load the Product Master Data table. This time, the preview window (Figure 6.14) shows that the column names have not been recognized correctly and have been assigned instead with some default identifiers (*Column1*, *Column2*, and so on). This triggers the need for some minor transformation before using this table in our data model. Hence, this time we click on the second button at the bottom, **Transform Data**:

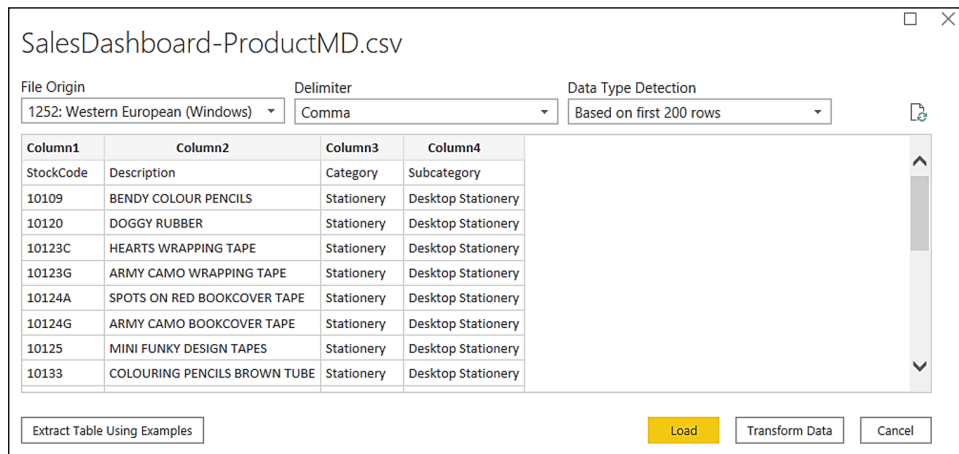


Figure 6.14: Preview of the Product Master Data table: column headers were not recognized

3. By clicking on **Transform Data**, we ask Power BI to add some transformation steps to the query that injects the Product Master Data table: this is a job for **Power Query Editor**, whose window will now appear. In this specific case, the only necessary change is to promote the first row read in the CSV file to be used as headers of the table. To do so, click on the **Use First Row as Headers** button, which you can find on the **Home** ribbon menu (see the arrow in *Figure 6.15* as a guide). Once done, the list of the **Applied Steps** on the right is enriched with a **Promoted Headers** step: this means that every time new data gets loaded, Power BI will execute this step as part of the query. We can click on the **Close & Apply** icon on the top left and move on:

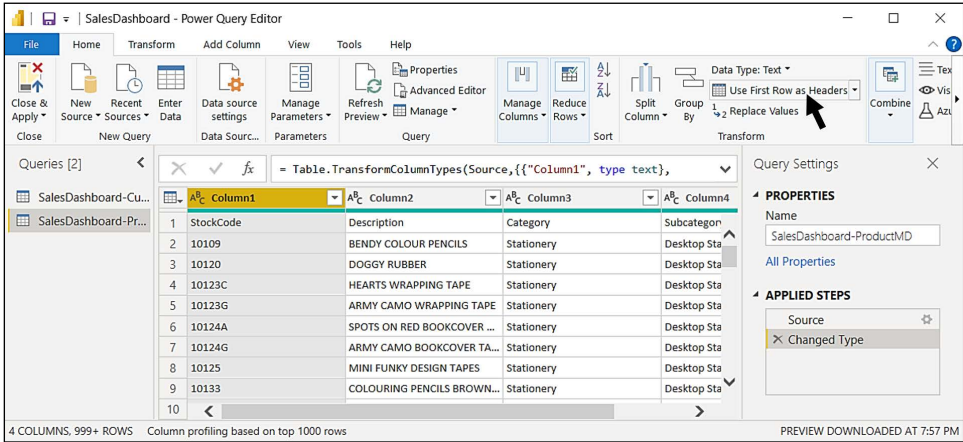


Figure 6.15: Power Query Editor for the Product Master Data table.
We need to use the first row as headers

- The table containing the transactions is stored in an Excel file. To load it, you can either click on the usual **Get Data** and then select **Excel** or click straight away on the **Excel** icon appearing in the **Home** ribbon menu. You will be presented with a dialog that lets you preview the tables contained in the various worksheets of the Excel file (Figure 6.16):

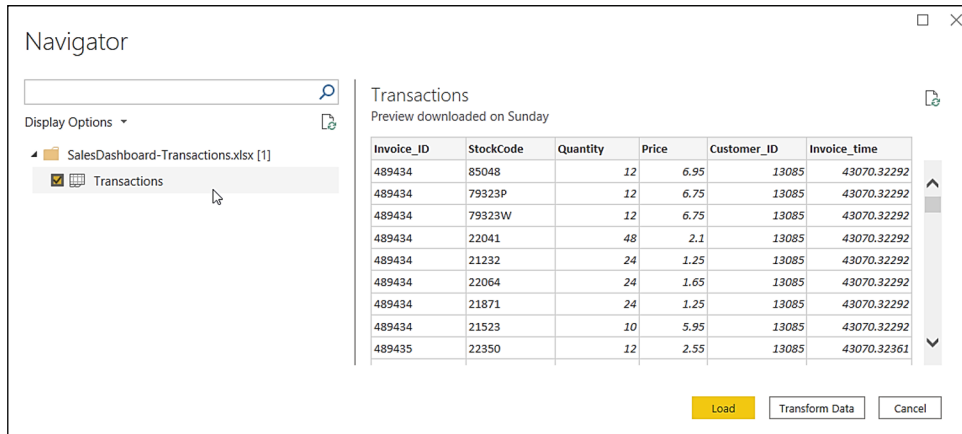


Figure 6.16: Excel preview dialog: select the tables you want to import from the Excel file

Also, in this case, we need to add some transformations to the query, so tick the **Transactions** box on the left and then click on **Transform Data**, which will open **Power Query Editor**.

As we look at the preview of the **Transactions** table, we notice that at least two transformations are required to make it usable. Firstly, the *Invoice_time* column does not look right, as it has not been recognized as a Date & Time field. Secondly, since most of our business needs require the analysis of revenues and we only have unit *Price* and *Quantity* for each transaction, we need to implement a simple mathematical formula to obtain the revenues by multiplying the two columns. Let's see how **Power Query Editor** can help us with both.

5. If you look at the headers of the spreadsheet view, you will notice that beside every column title, on the left, you can find an icon representing its data type. For example, the **ABC** icon near *StockCode* tells us that it is a text string, while the **1.2** near *Price* indicates it is a decimal number. If you click on any of these icons, you will see a pop-up menu (Figure 6.17) from where you can force a different data type. Power BI will attempt the conversion to the type you wish. Let's do this with the *Invoice_time* column: click on its type icon on the left and select **Date/Time** (if prompted with a window asking whether to replace the current query step or add a new one, select the first option). You will notice that the spreadsheet view now shows dates and times, which is exactly what we wanted:

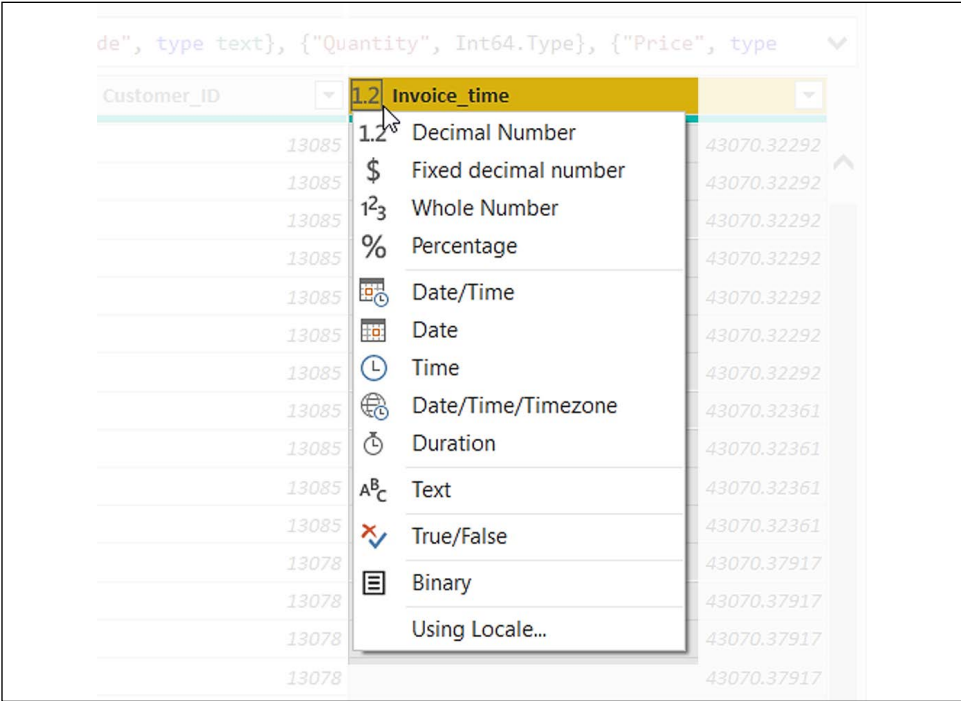


Figure 6.17: Column type selection from Power Query Editor: convert columns by making the right selection

- The next step is to add a new column that hosts the revenues generated by each transaction. To do so, go to the **Add Column** ribbon menu and then click on the **Custom Column** icon. You will obtain the window shown in *Figure 6.18* from where you can define your expression, similar to what we did using the **Math Formula** node in KNIME. The available columns are displayed on the right: double-click on *Quantity* to make it appear in the **Custom column formula** box, then just type `*` with your keyboard, and – finally – double-click on the *Price* column on the right. By doing so, you will obtain the expression `[Quantity]*[Price]`, which is what we needed to calculate value sales. You can now specify its header by typing *Sales* in the **New column name** box on the top, then click on the **OK** button:

Custom Column

Add a column that is computed from the other columns.

New column name

Sales

Custom column formula ⓘ

= [Quantity]*[Price]

Available columns

- Invoice_ID
- StockCode
- Quantity
- Price
- Customer_ID
- Invoice_time

<< Insert

[Learn about Power Query formulas](#)

✓ No syntax errors have been detected.

OK Cancel

Figure 6.18: Add a custom column in Power BI: define your math formula using columns as variables

You will notice with pride that the newly calculated column *Sales* appears on the right of the spreadsheet view. However, its data type is still undefined, as you will find displayed on the left of the title an ambiguous **ABC123** icon. You can quickly fix this by clicking on the icon and selecting **Decimal Number**.



Always declare the data types of your columns. By doing so, columns can be used as optimally as possible in your dashboard visualizations, and, also, they will get appropriately aggregated. Numeric columns are, by default, aggregated in Power BI by summing values across, but you can change the aggregation function for each column at any time. To do so, go to the **Data** view (see *Figure 6.2* to find it), select a column, and use the **Summarization** drop-down menu you find in the **Column tools** ribbon menu (for numeric columns, you can go for **Sum**, **Average**, **Minimum**, **Maximum**, **Count**, and **Distinct Count**).

Once you have completed this step, you will see that with the help of Power Query Editor, you created a table with seven columns, as displayed in *Figure 6.19*. In the **Applied Steps** panel on the right, you find a sorted list of all the transformations that need to happen (like sourcing the data stored in Excel, adding columns, changing their types, and so on) to produce the table you want to use within your dashboard. You can click on each step to "go back" to that phase of your transformation routine. You can also remove any step (click on the **X** icon on the left that appears when you select the specific step) or edit it: for instance, you can change the definition of the custom formula we added by clicking on the cog icon on the right of the step called **Added Custom**:

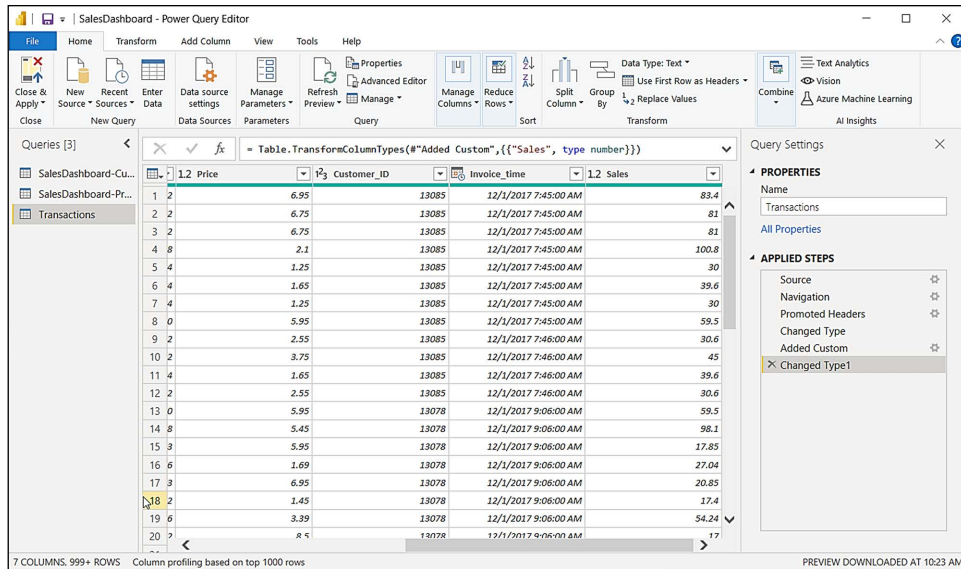


Figure 6.19: The full query definition for your Transactions table: Power Query Editor is your ally for repeatable data transformations in Power BI



Although most of the transformations are typically defined when adding new data sources, you can edit your queries later by reopening the Power Query Editor view (click on the **Transform Data** icon from the **Home** ribbon menu).

To proceed further, click on the **Close & Apply** icon in the top left, and we are ready to go to the next phase of our dashboard creation process: the definition of the data model.

7. If you go to the **Model** view (click on the third icon in the bar to the left of your Power BI main window), you will find a familiar picture: the ER model of the database made of our three tables (go back a few pages and check *Figure 6.8* to see a screenshot of the data **Model** view). Power BI has automatically recognized the relationships across the tables and built the connections without our intervention. Of course, in this case, it was rather simple since the matching columns had precisely the same name and type. In any case, we can manually declare or edit any relationship across tables by using the **Model** view. We can "drag and drop" one column in a table to another column on a different table to create a new relationship. To edit an existing connection, you can double-click on the line linking two tables. Click on the relationship between ProductMD and Transactions: you will obtain the **Edit relationship** view displayed in *Figure 6.20*. The drop-down menus let you select the two tables to be connected, while you can specify the matching columns by clicking on the preview tables displayed in the middle. In our case, we notice that the *StockCode* column is highlighted in both tables, which is what we wanted. The **Cardinality** drop-down menus at the bottom left let you select the type of relationship (**Many to one**, **One to one**, **One to many**, or **Many to many**): by doing so, you are telling Power BI how many rows in the first table should match with how many rows in the second table. In this case, we go for **Many to one** since multiple transactions (first table) can correspond to the same product, and so to one single row in the Product Master Data table (the second table). You can keep the **Cross filter direction** drop-down menu to its default value (**Single**, which is recommended, for performance reasons, in the large majority of cases). We did not have to make any changes as all the default options worked well in this case: click on **OK** and move on:

×

Edit relationship

Select tables and columns that are related.

Transactions

Invoice_ID	StockCode	Quantity	Price	Customer_ID	Invoice_time	Sales
489875	85049E	1	1.25	17841	12/2/2017 3:41:00 PM	1.25
489875	37464	1	1.25	17841	12/2/2017 3:41:00 PM	1.25
489875	21098	1	1.25	17841	12/2/2017 3:41:00 PM	1.25

SalesDashboard-ProductMD

StockCode	Description	Category	Subcategory
20671	BLUE TEATIME PRINT BOWL	Home	Kitchen
20672	PINK TEATIME PRINT BOWL	Home	Kitchen
20688	BEAUTY SPEED EXPRESSO CUPS	Home	Kitchen

Cardinality

Many to one (*:1)

Cross filter direction

Single

☒ Make this relationship active
 ☐ Apply security filter in both directions

☐ Assume referential integrity

OK

Cancel

Figure 6.20: Manage a relationship across tables in a data model: which columns should be matching?



In Power BI, the active relationship between two tables can only be defined through one pair of matching columns. If you want to match multiple columns across two tables, you need to go through this simple workaround: go to **Power Query Editor** (click on the usual **Transform Data** icon), select the two or multiple columns you want to use collectively as a single key, and click on **Merge Columns** from the **Transform** menu. Do this on the other table as well, and you will then be able to create a relationship across the merged columns.

8. Check the other relationship between Transactions and the Customer Master Data table by double-clicking on the line linking the two tables. Make sure that the matching columns are *Customer_ID*, that **Cardinality** is **Many to one**, and that **Cross filter direction** is **Single**. Then click on **OK**, and your data model is ready to go!

At this point, we are ready to move to the last and, for some, most exciting part: designing your dashboard by building its visuals one by one. One mistake many people tend to make the first time they create any data dashboard is to jump into crafting colorful charts without having a reasonably clear idea of what they are after. My advice is to always take some time to take a step back, think about the specific business needs you have, and put together a simple *sketch* of what the dashboard should look like. Think about your end users' questions and how to effectively guide them through the answers they are looking for. If you have the opportunity, show your sketch (or a digital *mock-up* built with any software you feel comfortable with) to some of the end users to get their first reaction. You will find that the time you invest early on in planning for your dashboard (no need for details at this stage, just a high-level view of its content will suffice) always pays its dividends as you avoid some expensive (and frustrating) rework later.



When you work on large descriptive analytics initiatives (like building from scratch the business intelligence application layer of a company), the planning phase becomes even more crucial. The advice, in this case, is to keep an *agile* mindset and work in small (but always well-planned) iterations where you add functionalities progressively at every step. In this case, the other advice is to make sure you engage your business partners (representing the end users of the dashboard) early on and *co-design* the capability together with them instead of presenting the end product when it is done.

In the case of our sales dashboard, we will need to pull together an overview page that will have to contain sales split by category and subcategory. A bar chart and a treemap can give a good indication of the relative size of each product element (don't worry about the visualization types at this point; we will learn how to pick the best ones in the next chapter). A line chart will be useful to show evolution over time, and it will be nice to visualize the distribution of sales on the world map to hint at any regional patterns. Lastly, our users will need to be able to select specific year ranges and whether to see the domestic sales only (generated in the UK), or the international ones, or both: some slicers will have to be added to give a simple way to filter the data of interest. By keeping the end users' needs in mind, we put together the sketch design that you see in *Figure 6.21*: we are, by choice, far from perfection at this stage, but at least we can confidently move on to build visuals knowing what we are aiming for:



Figure 6.21: A rough sketch of the Sales Overview dashboard page: it's better to decide what data you need to visualize to meet your business needs before creating any chart

- Go to the **Report** view (first icon on the vertical bar at the left of the main window). In the **Visualizations** panel on the right (the ones with all the icons showing the different chart types), click on the icon with the horizontal bar chart (its tooltip label, obtained when you leave the mouse for a second, reads **Stacked Bar Chart**). An empty box pops up on the report page: this is a placeholder for our bar chart. We now need to tell Power BI what data we want this bar chart to show. As shown by the arrows in *Figure 6.22*, we need to drag the columns available in the data model (listed in the **Fields** panel on the right) and drop them to the properties panel of the visualization. More specifically, the column *Category* (open the Product Master Data group to find it) should end up in the **Axis** box on the properties panel, while *Sales* (in the Transactions table) should be dropped to the **Values** box. As soon as you do that, the bar will magically appear on the left, and your first Power BI visual is alive! We could change the *look and feel* of the newly created visual (changing colors, titles, fonts, labels, and so on) by using the **Format** panel (with the paint roller icon), but, for now, we focus on building the rest of the dashboard, and we move on to the following chart:

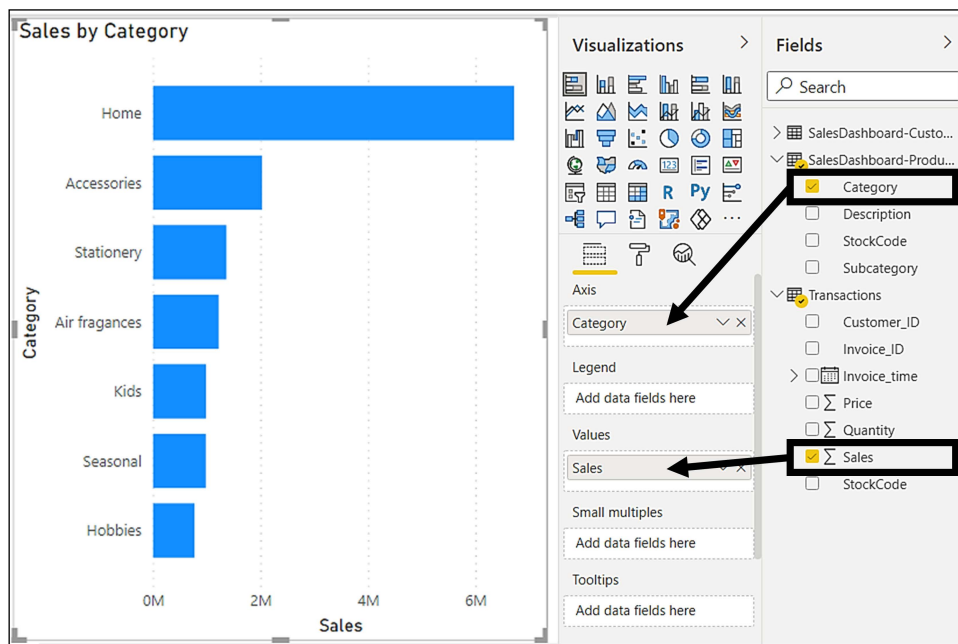


Figure 6.22: Building a bar chart in Power BI: pick the columns to be used for the axis categories and the height of the bars

10. As we follow through with our sketch, we add now a treemap: this will show the relative size of each category/subcategory combination in terms of sales. Click on an empty space in the report page and then on the **Treemap** visualization icon. To build the visual, drag and drop the fields of interest to the right property boxes (Figure 6.23): *Category* should go to **Group**, *Subcategory* to **Details**, and *Sales* to **Values**. By now, you should have the essence of building charts in Power BI: we decide what type of visual we need, and then we connect the data columns with the different property fields available for that type of chart:

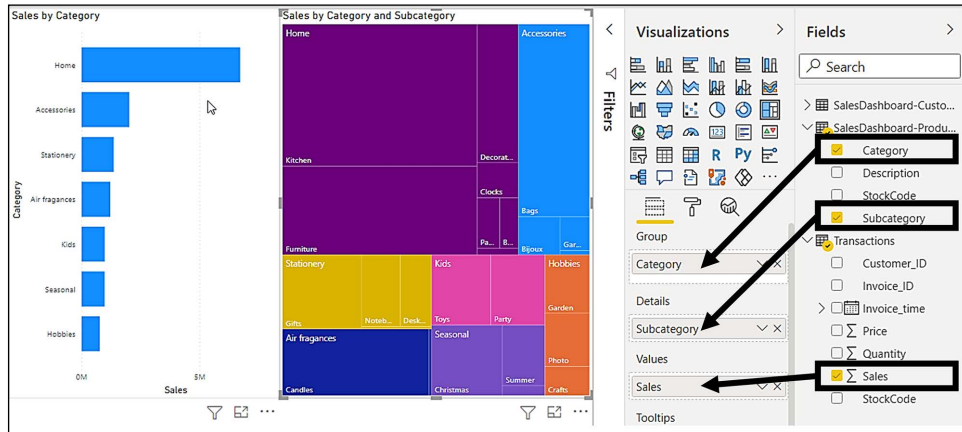


Figure 6.23: Treemap visualization: show the relative size of each group and subgroup



You will notice that every chart has got some gray handles at its edges and corners. Use them to resize it and move the chart as needed to fit it to the position you like. Considering the sketch in mind, start putting the visuals in the place where they belong.

11. The next visual to add is a line chart that shows the sales evolution over time. Click on an empty space in the report space and then on the **Line chart** icon. You can use *Invoice_time* as the column to control the chart **Axis** while *Sales* will determine the **Values** to be visualized. Power BI uses by default only the top level of the time hierarchy: this means that, at first, the visual will show the aggregate sales by year, displaying only three values, one for each year in the 2017-2019 range. Since we would like to show the evolution by month, we need to expand to the levels below in the hierarchy by clicking on the bifurcation arrow on the top right of the visual (follow *Figure 6.24* as a guide). The next level shows the split by year/quarters, but this is not enough as we want to move to years/months, so you will need to click on the icon once again: at this point, the title of the chart will become **Sales by Year, Quarter and Month**, indicating that we obtained what we were after:

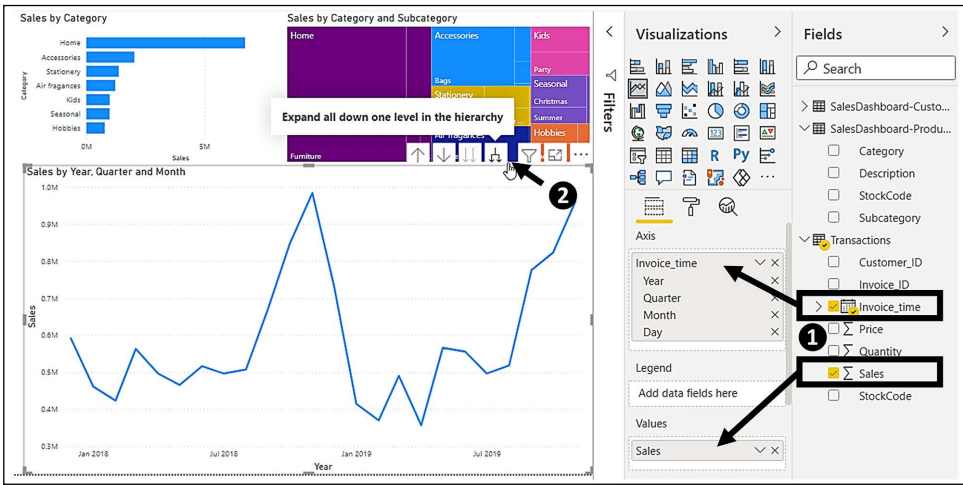


Figure 6.24: Building a line chart to show the evolution over time: you can click on the arrow icons at the top right of the chart to navigate through your year/quarter/month/day hierarchy as needed

12. The next chart in our Sales Overview page is the world map, showing sales by country. The good news is that Power BI automatically recognizes geographical names in tables, making it very simple to create map charts. Click on an empty space in the report and then on the **Map** visualization (the icon looks like a globe; don't get confused with **Filled map**, which shows a sketched map of the United States instead). As done in Figure 6.25, drag the column *Country* (in the Customer Master Data table) to the **Location** box and then use *Sales* to control the **Size** of the bubbles. You now have a world map showing where revenues originate from: not surprisingly, the biggest bubble is in the United Kingdom, given the prevalence of domestic customers:

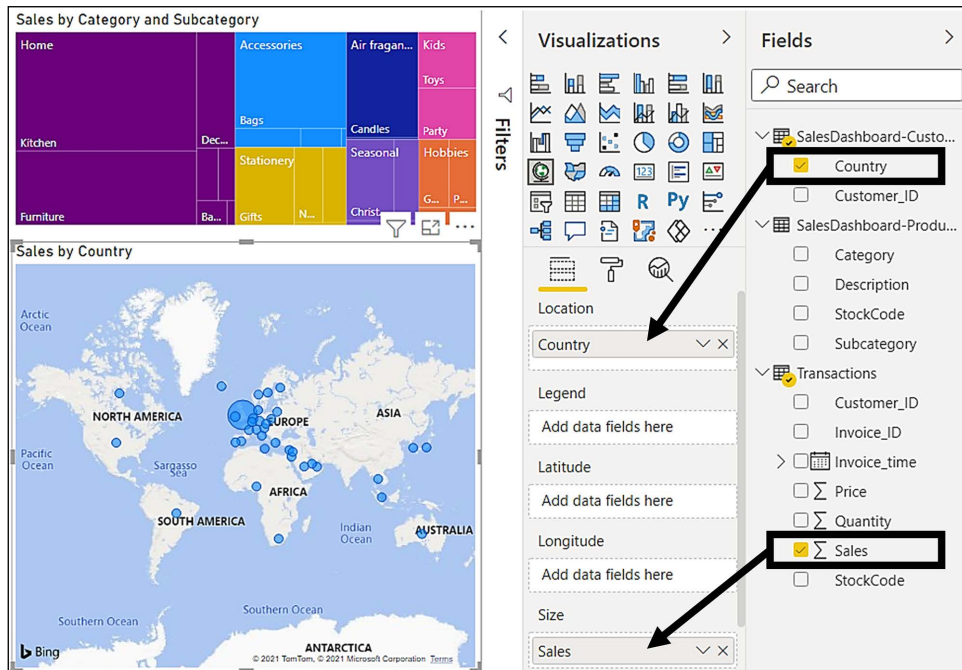


Figure 6.25: A map chart in its full glory: Power BI recognizes locations like countries, states, and city names, making it simple to build maps enriched with data



In the map visualization (as in any other chart in Power BI), when you keep the mouse on a visual element (in this case, a bubble on the map), you get a tooltip label that gives you, by default, the precise value referring to that specific element (like the total sales originated in a particular country). Many chart types allow you to control what to show in a tooltip by adding fields to the **Tooltips** box.

13. It is time to add a couple of slicers to let the user select what to visualize on the page. The first one will control the time dimension: we want to let the users limit all the visuals on our Sales Overview page to a specific time frame so that they can assess sales by category, subcategory, or country in a given time period. To do so, it will be enough to add a **Slicer** visualization and drag the *Invoice_time* column to the **Field** box, as shown in *Figure 6.26*. Slicers are very popular in Power BI: you can add as many as you need and mix multiple fields into one single slicer to allow multi-level selection. For instance, we could combine *Category* and *Subcategory* together so that users could navigate the entire product hierarchy as they decide what to focus on:

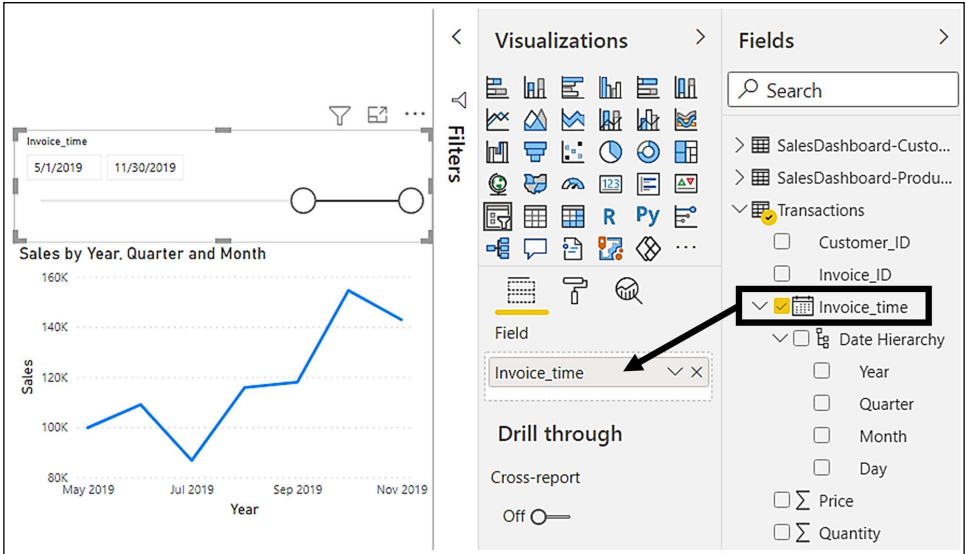


Figure 6.26: Slicers in Power BI: visually filter your data so you can focus on what matters most.

14. According to our sketch, our second slicer should discriminate between domestic and international sales. Since we do not have a column that specifies which group a customer belongs to, we need to add a new column that fulfills the need. Power BI provides for **Conditional Columns**: these are calculated columns that apply a given list of logical rules (like a set of *if-then* clauses) to determine the value for each row. You will notice that this is similar to what we used to do with the **Rule Engine** node in KNIME. To add conditional columns in Power BI, we need to go back to the Power Query Editor window by clicking on the **Transform Data** icon on the **Home** ribbon menu. After selecting the Customer Master Data query from the list on the left, we can add our column by going to the **Add Column** ribbon and clicking on **Conditional Column**. You will be prompted with a window like the one shown in Figure 6.27: after choosing the name of the new column using the textbox on top, you can define one or more *if-then* clauses, leveraging operators such as *equals*, *is greater than*, *includes*, and so on. You can append alternative conditions by clicking on the **Add Clause** button and also specify the default value to use if none of the conditions is met (the **Else** box, at the bottom). In our case, we just want the new column (which we can call *Type*) to show Domestic if *Country* is United Kingdom and International in any other case:

Add Conditional Column

Add a conditional column that is computed from the other columns or values.

New column name
Type

Column Name	Operator	Value	Output
If Country	equals	United Kingdom	Domestic

Add Clause

Else
International

OK Cancel

Figure 6.27: Adding a conditional column: define a set of logic clauses to control the column's values

15. Now that we have the new column *Type* available, we can add a new slicer following precisely the same procedure we saw for the time slicer and dropping the new conditional column in the **Field** box.

16. Let's equip our page with a title by going to the **Insert** ribbon menu and clicking on the **Text box** icon. We can write any text we like (in this case, we enter the page title, which is **Sales Overview**) and play with the font size and type to make it look good. By dragging all visuals around and rearranging them according to our original sketch, we obtain the page shown in *Figure 6.28*:

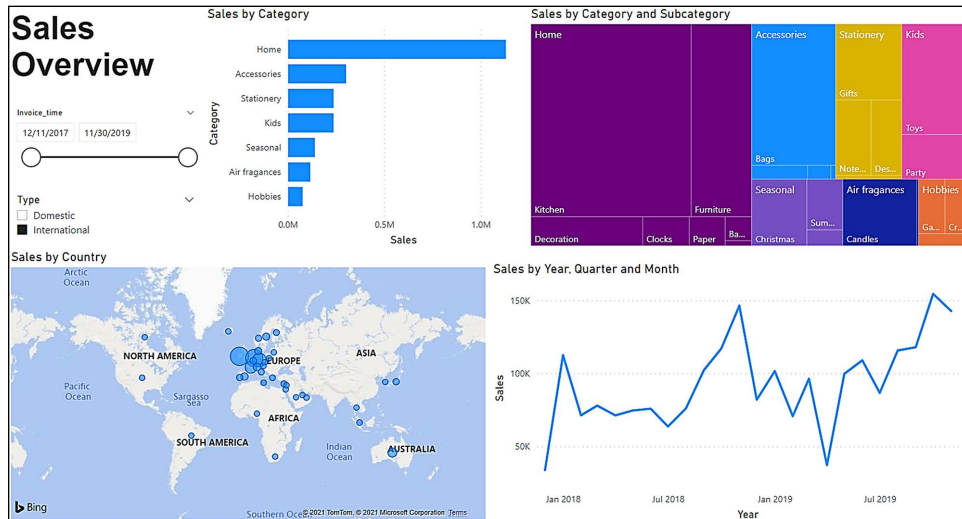


Figure 6.28: Sales Overview page: not bad at all for being our first Power BI creation!

This page is already compelling in its simplicity: it includes several visuals showing the key dimensions of interest within our online retail business. The slicers enable the users to filter their analysis within a specific time frame or country scope. Very importantly, these visuals are all connected with each other: the users can interact with a visual element, causing the other visuals to rearrange their scope. For example, try to click on any bar in the first category bar chart: the line chart will show the sales evolution for that specific category, and the world map will rearrange the size of the bubbles considering only the revenues generated within that particular country. If we select a subcategory from the treemap on the top right, the same will happen, and all other visuals will show the details for that subcategory. This is pretty powerful as we are letting the users discover the data autonomously, customize the views as they like, and, in a way, cook their own data exploration experience versus feeding them with a ready-made meal.



In Power BI, you can select multiple elements at once by keeping the **CTRL** key pressed. Try keeping the **CTRL** key pressed and click on two different categories in the bar chart: the sales of those two categories will be aggregated together in the line chart. You can also use this feature to slice the data across visuals. For example, click on a bubble in the map chart and, while keeping the **CTRL** key pressed, click on a subcategory from the treemap: the line chart will now show the time evolution of sales for that subcategory in the specific country you selected from the world map.

Since the charts interact with each other, we might need to fine-tune and modify their dynamic behavior. For instance, if you click on any category on the bar chart, the treemap will *highlight* your selection, meaning that all other categories will have their color faded. What if, instead, we want the treemap to react differently and, instead of highlighting the category selected in the bar chart, we want it to show only that category and filter out all others? Let's take this challenge as an opportunity to see how to change the interactions behaviors across visuals in Power BI.

17. Go to the **Format** ribbon menu (it appears when you select any chart) and click on the **Edit interactions** icon on the left. Select the visual that *controls* others (we can call it **antecedent**) by clicking on it. In our case, let's click on the bar chart showing sales by category. As we do so, three icons will appear in the top right of the treemap icon (make sure you resize it accordingly to be able to see the three icons popping up). By clicking on these icons (see *Figure 6.29*), you will decide how you want the treemap to respond when a selection on its antecedent occurs: the **Filter** icon will make that visual filter out any other data and "focus" on the selection; the **Highlight** icon (which is the default behavior) will cause the visual to emphasize the selection applied on the antecedent by coloring it differently; the **None** icon will break the connection and make the treemap completely "independent" from any selection on the bar chart. In our case, we click on the **Filter** icon (this is the behavior we are after) and then click again on the **Edit interactions** icon to get back to the normal editing mode:

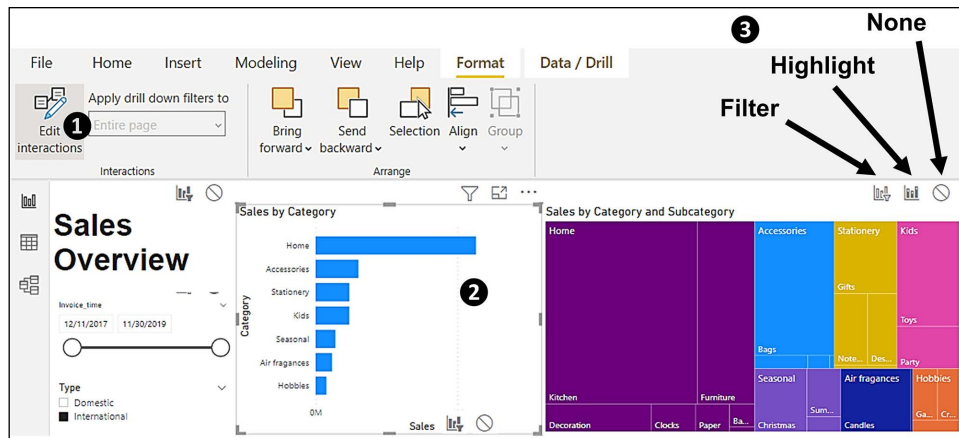


Figure 6.29: Three steps for changing the interaction behaviors across visuals: click on Edit interactions, select the antecedent visual, and then click on the resulting behavior icon (filter, highlight, or do nothing) on all subsequent visuals

Not all visual types allow the three types of interactions we have seen in this example. For instance, line charts only allow for **Filter** and **None** interactions.

By looking at the original set of requirements, we realize that we would need a few more visuals to complete our endeavor. Both marketing and category managers need to go beyond the existing visual overview and access some more detailed tabular exhibits. We decide to complete our dashboard by adding a page, called Sales Details, equipped with a handful of tables that match our remaining needs.

18. Click on the **+** sign at the bottom of the **Report** view: this will add a blank new page to your dashboard. You can rename every page by right-clicking on its tab and selecting **Rename Page**.
19. Let's add a **Matrix** visualization to the new page (don't get confused with the **Table** visualization, which we will use later). Now, add the following fields to the various boxes: put *Category* first and then, underneath, *Subcategory* in the **Rows** box. After that, drag *Invoice_time* to the **Columns** box, and, finally, put *Sales* as **Values**.

20. At this point, the table shows only the top level of the product and the time hierarchy (like in *Figure 6.30*), as this is its default behavior. To make it useful for our business users, we need to expand those hierarchies by clicking on the bifurcation arrow at the top right of the visual (see step 2 in *Figure 6.30*). Once you have done it for rows, use the **Drill on** drop-down menus to select columns (step 3 in *Figure 6.30*) and click again on the bifurcation arrow (like we did earlier for rows). You have now obtained quite a rich table, showing quarterly sales by category and subcategory:

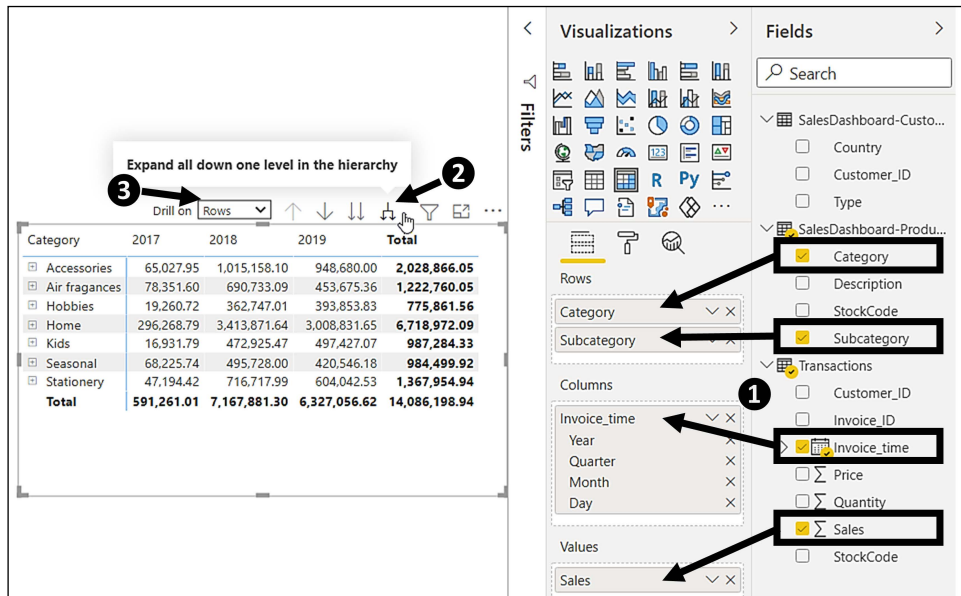


Figure 6.30: Building a matrix visualization: select the fields to use and decide the aggregation levels for both rows and columns

21. To make the matrix chart more accessible, we can fine-tune a couple of settings to remove some unneeded numbers. Go to the **Format** panel (the one having a paint roller icon), scroll down in the list of options, open the **Subtotals** view, and switch off **Column subtotals** as we don't need the yearly sums (see step 1 in Figure 6.31). Then scroll down and open the **Field formatting** view: from here, set **Value decimal places** to 0, so we remove all those unneeded decimal values, which only create clutter (step 2 in Figure 6.31):

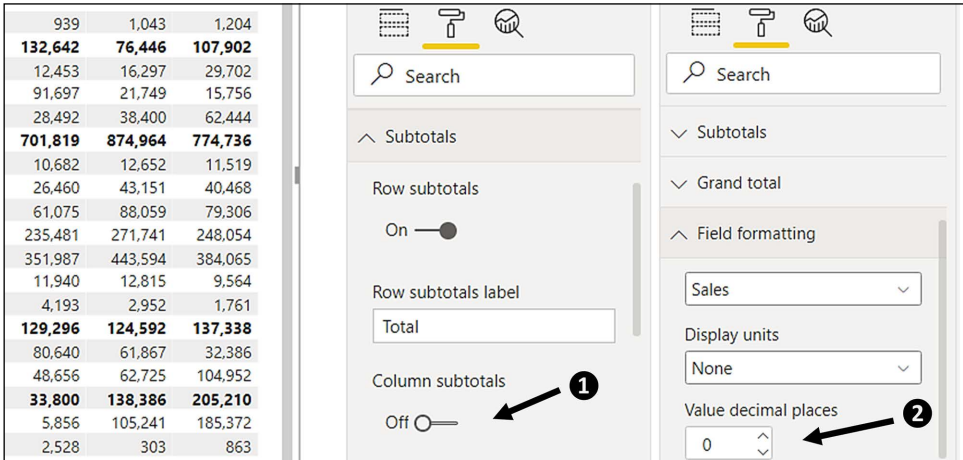


Figure 6.31: Removing the clutter from the matrix visual: deactivate columns subtotals and remove decimal digits

22. Let's add a new visual to give users the ability to focus on domestic or international sales if needs be. This time, add a **Donut chart** and set its fields as follows (we don't need to look at any screenshots as this procedure has hopefully become pretty familiar by now): let's use the column *Type* as **Legend** and *Sales* as **Values**.
23. To complete our detailed view, we shall show aggregated sales by country and by article. Let's start with the country: add a **Table** visual and drag and drop the columns **Country** and **Sales** directly into it (just make sure you follow the specified order, first **Country** and then **Sales**). By clicking on the header of *Sales* in the new table visual, the sales by country are sorted by decreasing revenues, which is what we were looking for.

24. To add the article details, let's implement another **Table** visual and then drag and drop the following columns into it, following this order: *Description*, *Quantity*, and *Sales*. Similarly, as done before, click on the *Sales* header in the table visual to put the best-selling articles at the top of the list. To avoid clutter, remove the decimal digits both here and in the previous table, using the **Field formatting | Value decimal places** selectors, as we did for the matrix view.
25. We should add a title to this page as well. Go to the **Insert** ribbon menu, click on the **Text box** icon and write *Sales Details* into it. For coherence, try to put the same font size and type you used for the first page:

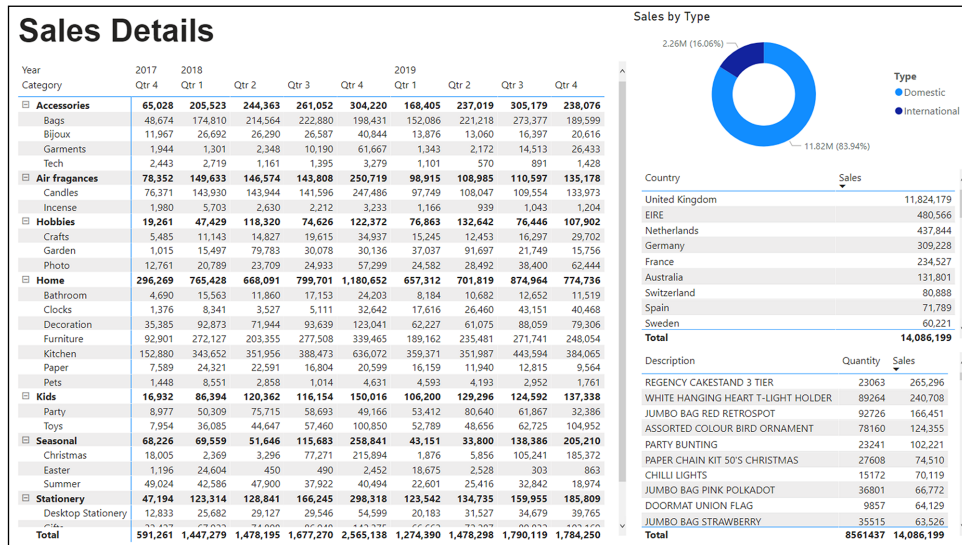


Figure 6.32: Sales Details page: our Power BI dashboard is now complete with the possibility to get to the granular country and product details

We are finally done: the second page you've just pulled together (see Figure 6.32) has nicely provided what we were missing. Users can now dive into subcategory/quarterly/country combinations (it will be enough to click on the headers of the tables to filter down to the combination they need, keeping **CTRL** pressed) and retrieve the top-selling articles within every segmentation of the business.

When you publish your new, shiny, two-page dashboard on the Power BI service (if you have an account, even a free one, go to the **Home** ribbon menu and click on the **Publish** icon to give it a try) and share its URL with your colleagues, you will suddenly become their hero! They can now access the data they need from any browser (see *Figure 6.33*) and autonomously deep-dive to the parts of the business they are most interested in:

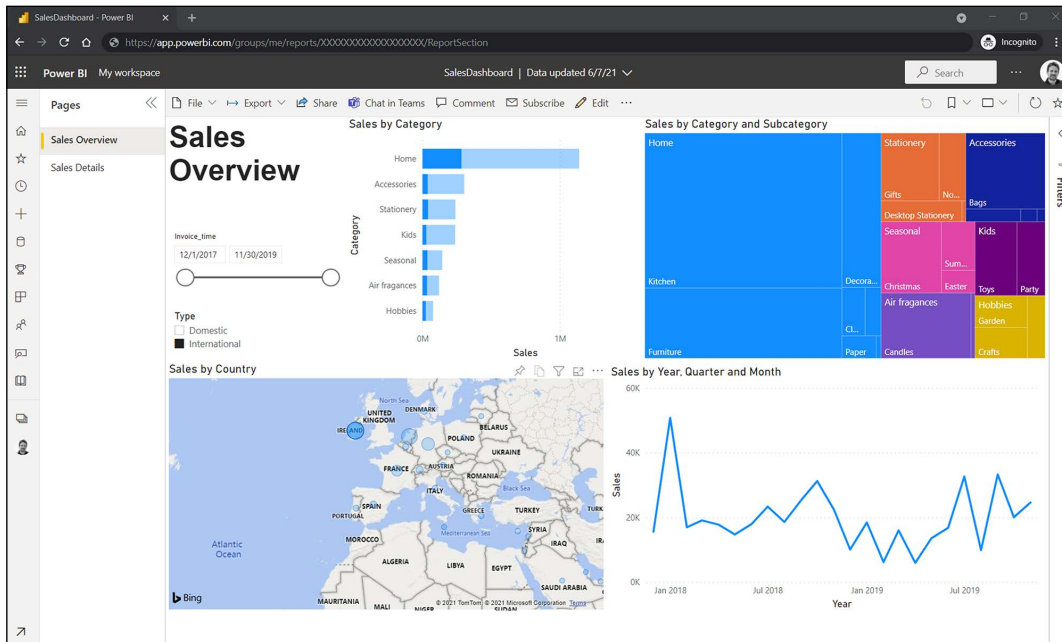


Figure 6.33: Browser view of the Sales Overview page once published on the Power BI service: our dashboard looks great from the cloud

When traveling, our colleagues can even check sales results while *on the go*: you can see in *Figure 6.34* how our dashboard looks in the Power BI app on a smartphone:

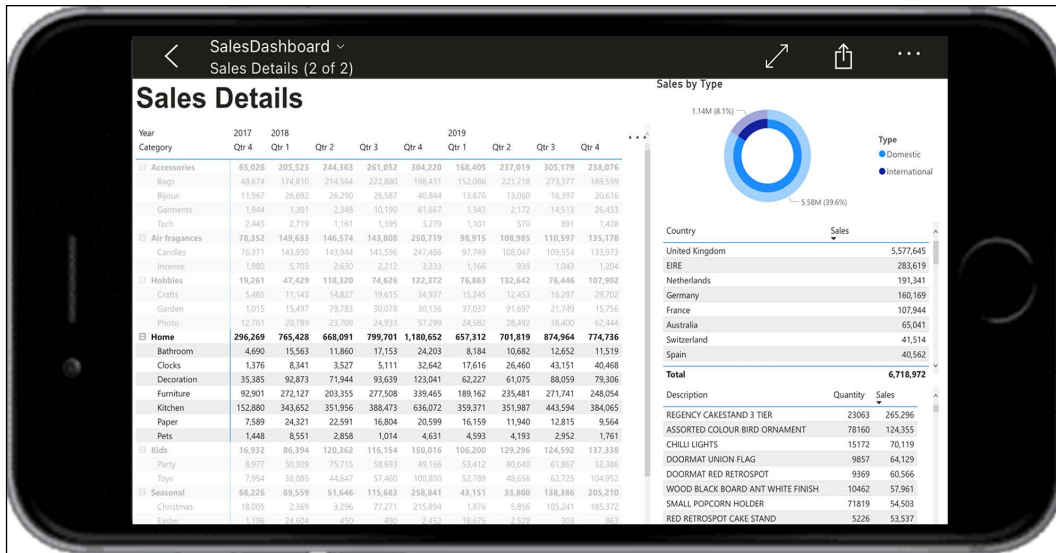


Figure 6.34: The Sales Details page accessed through the Power BI app for iPhone: you can access your data from virtually anywhere

The maintenance of the dashboard is relatively lightweight as the updating of all visuals can now happen even daily without much effort. The various queries we have defined know where to pull the data from and what transformation steps are required. When new data pops up, it will be enough to refresh the dashboard by just clicking on the **Refresh** icon on the **Home** ribbon menu, and the whole dashboard will be nicely up to date.

Not only have you finally democratized access to sales data in the company through a sustainable process that can now be repeated effortlessly, but you have also saved the financial analyst a lot of her precious time. She will not need to answer all the many "ad hoc" questions that she frequently used to receive and that forced her to set up painful manual extractions every time. As the data is now readily available, it is faster and more convenient for people to go and get what they need by themselves, giving back to our financial analyst the precious time she can use for more advanced analytics. It's a real *win-win*, and you should feel very proud of having enabled all this.

Summary

By completing this chapter, you have added another precious instrument to your data analytics practitioners' backpack: Power BI. You have learned how to load data and set up a routine of transformation steps required for cleaning that data and making it useful. You have seen how to define the relationships that connect all tables into a single and coherent data model. You have also seen what it means to build interconnected visuals that allow for human interaction. You have created a dashboard that will enable business users to explore the data they need when they need it, *fishing* for insights and unveiling business opportunities in complete autonomy.

Now, think about the amount of data that lies largely unutilized and inaccessible in company databases (or even in Excel files stored on individuals' laptops). The business opportunities that wait to be unlocked by making this data visually accessible through a simple dashboard are vast and, in many cases, reasonably at hand with tools like Power BI: what you have learned in this chapter can make you a hero to many.

In the next chapter, we will learn about some practical guidelines to sharpen our data visualization skills: the objective is to make the best out of every single chart we produce from now on.